

AMERICAN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Engineering & Computer Science ·



Steel Cut Optimization System

Presented by: Ahmad Abou Ilawi | Lama Al Sayed | Shaza Hamzeh | Hakima Bazzal

Advisor: Dr. Hasnaa Hussein Sleiman

Outline

01 Introduction

02 Tools

03 System Architecture & Logic

04 Program Flowchart

05 Program Execution Demo

06 Discussion — Inputs · Processing · Outputs

07 Challenges

08 Results & Conclusions

Introduction — Problem Statement

1.Steel cut Optimizer is designed to minimize material waste and improve cutting efficiency

2.Utilizing **C++ arrays, loops, and kerf-aware formulas** for accurate plate–piece fit calculations

3.The system's aim is to provide **optimized plate selection, rotation support, and instant cost quotation**

Tools

C++

Language

ISO C++

[ ]

Arrays

Static arrays for data storage

Loops

Optimization & input loops

Arithmetic

Area, efficiency & cost formulas

Validation

Range & sign checks on all inputs

Program Execution — Sample Run

▶️ INPUT

```
length and width in m, thickness is in mm
Enter the cutting blade thickness (kerf): 1
Enter the total number of unique stock plate types available (Maximum 100): 2

--- Stock Plate Type 1 ---
Enter numeric ID for this plate type: 0
Enter width of this plate: 1
Enter length of this plate: 2
Enter available stock quantity for this plate: 10

--- Stock Plate Type 2 ---
Enter numeric ID for this plate type: 1
Enter width of this plate: 0.5
Enter length of this plate: 1
Enter available stock quantity for this plate: 10

Enter the total number of unique required piece sizes needed (Maximum 100): 2

--- Required Piece Type 1 ---
Enter required width: 0.7
Enter required length: 0.9
Enter required quantity: 5
Is rotation allowed for this piece? (Enter 1 for YES, 0 for NO): 1

--- Required Piece Type 2 ---
Enter required width: 1
Enter required length: 0.6
Enter required quantity: 6
Is rotation allowed for this piece? (Enter 1 for YES, 0 for NO): 1
```

◀️ OUTPUT

```
============================================================
                    OPTIMIZATION RESULTS
============================================================
Recommended Plate ID: 0
Recommended Plate Size: 1 x 2
Number of Plates Needed: 4
Total Pieces Produced: 11
Total Waste Area: 1.25
Efficiency Percentage: 84.375%
Remaining Stock Quantity of this Plate: 6
============================================================
```

▶️ INPUT

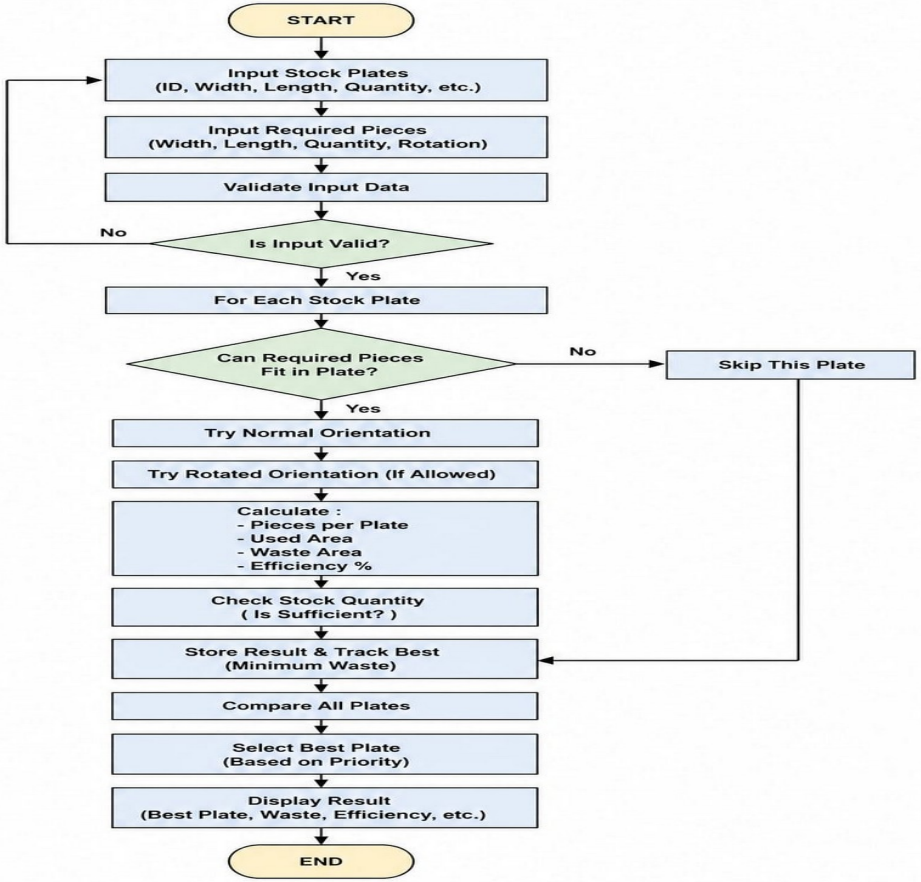
```
============================================================
              FINANCIAL COST & QUOTATION MODULE
============================================================
Enter the material cost per square unit of area: 1.25
Enter the cutting labor fee charged per plate: 3
Enter the sales tax rate percentage (e.g., 15 for 15%): 5
```

▶️

```
--- Financial Summary breakdown ---
Total Material Cost:      $10
Total Labor/Cutting Cost: $12
------------------------------------------------------------
Subtotal:                 $22
Tax Amount:               $1.1
------------------------------------------------------------
FINAL CUSTOMER INVOICE:   $23.1
============================================================
```

◀️ OUTPUT

Program Flowchart



Discussion — Inputs · Processing · Outputs

INPUTS	PROCESSING	OUTPUTS
Blade kerf thickness	Kerf-adjusted fit formula	Recommended plate ID & size
Plate: ID	Normal & rotated orientation test	Plates needed + remaining stock
Piece: Width · Length · Qty	Stock sufficiency enforcement	Total pieces & waste area
Rotation flag per piece (0/1)	Best plate: min waste priority	Efficiency percentage
Cost params: rate · labor · tax	Tiebreaker: max efficiency	Full itemized invoice

Challenges

01	**Debugging Error** PROBLEM	Faced logical mistakes in fit formula and loops; solved by step-by-step testing
02	**Time Management** PROBLEM	Writing and testing all modules took longer than expected; solved by splitting tasks among team members.
03	**Code Efficiency** PROBLEM	Editing and rewriting code from scratch multiple times to make it more efficient.

Results & Conclusions

- Accurate plate area and cost calculations.
- Reduced manual calculation errors.
- Replace arrays with dynamic data structures (such as vectors) to handle larger data-sets.
- . Apply Object-Oriented Programming (OOP) to improve code organization , and scalability.

Thank You!

We are ready for your questions.

Project
SmartCut Plate Optimizer

Language
C++ Console Application

Course
CSI205L